# Threat Monitoring & Alert Management Platform

**Developer:** Vicky Kumar
**Date:** December 25, 2025
**GitHub Repository:**
https://github.com/imvickykumar999/Threat-Monitoring-Alert-Management-Platform

---

## 1. Overview

### Project Overview

The **Threat Monitoring & Alert Management Platform** is a Django REST API–based backend system designed to ingest security events, automatically generate alerts for high-risk activity, and provide structured alert lifecycle management. The platform supports JWT-based authentication, role-based access control, and scalable RESTful APIs suitable for SOC-style monitoring tools.

### Key Objectives

- Centralized ingestion of security events
- Automatic alert creation for High/Critical severity events
- Secure access via JWT authentication
- Clear separation of roles (Admin / Analyst)
- Production-ready architecture with Docker support

---

## 2. Features

- **JWT Authentication** with refresh tokens
- **Role-Based Access Control** (Admin, Analyst)
- **Event Ingestion API** with automatic alert generation
- **Alert Lifecycle Management** (Open, Acknowledged, Resolved)
- **Filtering, Ordering & Pagination** for APIs
- **Swagger / OpenAPI Documentation**
- **Dockerized Deployment**
- **SQLite (Dev) & PostgreSQL (Prod) Support**
- **Comprehensive Unit Tests**

---

# 3. Architecture

## 3.1 Database Schema

### *User Model*

Custom user model extending `AbstractUser`.

**Fields** - `username` – CharField - `email` – EmailField - `role` – CharField (`admin`, `analyst`) - Standard Django user fields

**Indexes** - `role`

---

### *Event Model*

Stores ingested security events.

**Fields** - `source_name` – Source system name - `event_type` – Type of event - `severity` – Low / Medium / High / Critical - `description` – Detailed event description - `timestamp` – Auto-generated timestamp

**Indexes** - `severity` - `event_type` - `timestamp` - `source_name`

**Behavior** - Automatically triggers alert creation for High/Critical severity

---

### *Alert Model*

Tracks security alerts derived from events.

**Fields** - `event` – ForeignKey to Event - `status` – Open / Acknowledged / Resolved - `created_at` – Alert creation timestamp - `resolved_at` – Resolution timestamp (nullable)

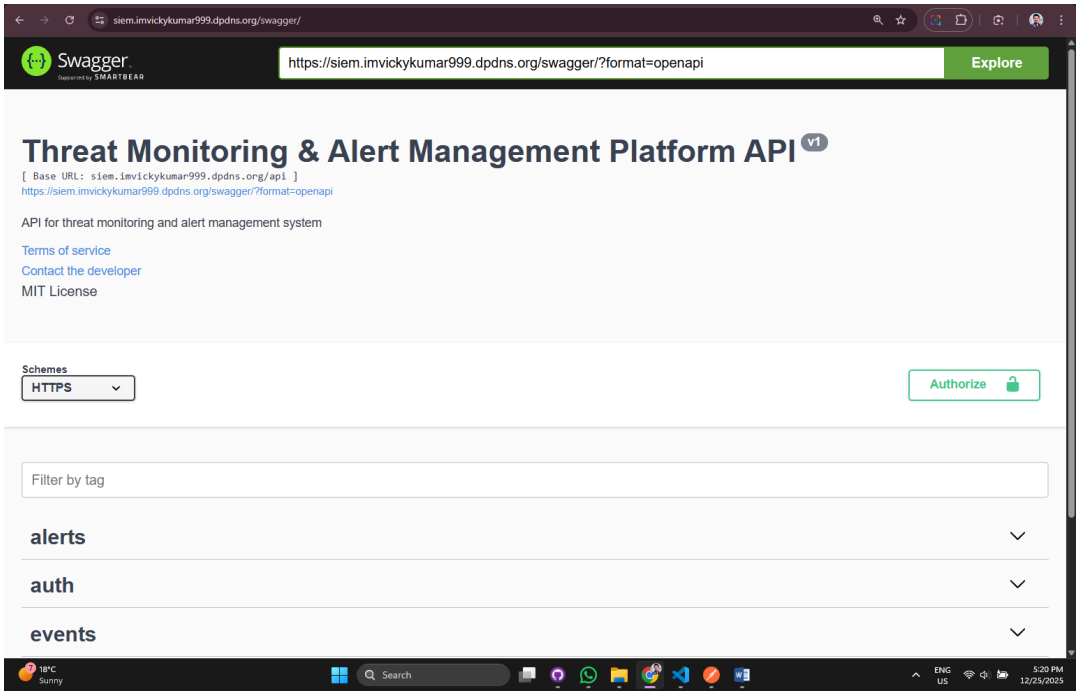
**Indexes** - `status` - `created_at` - `event`

**Relationships** - One Event → Many Alerts

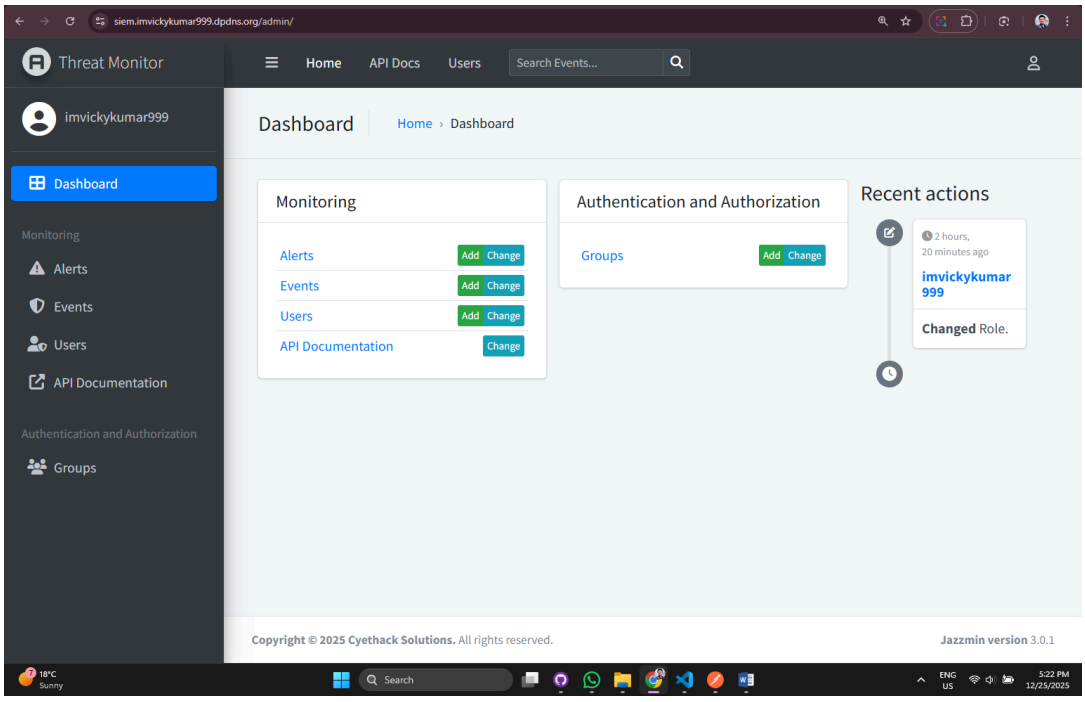
---

# 4. Screenshots

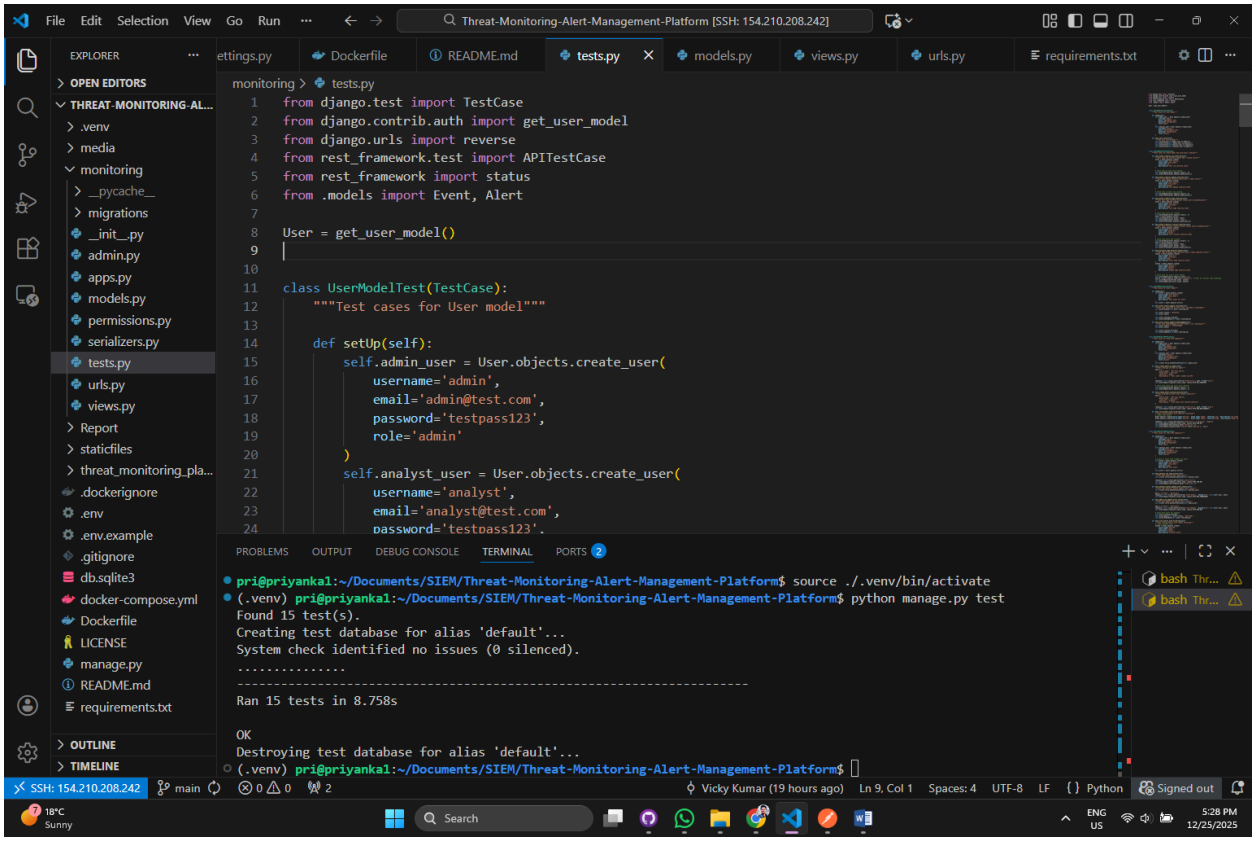
## 4.1 Swagger UI




## 4.2 Admin Panel



## 4.3 Tests Passed




# 5. Technology Stack

## Backend

- Django 4.2+
- Django REST Framework 3.14+
- Simple JWT 5.2+

## Database

- SQLite (Development)
- PostgreSQL (Production)

## Libraries & Tools

- django-filter
- drf-yasg (Swagger/OpenAPI)
- django-jazzmin (Admin UI)
- psycopg2-binary

- dj-database-url
- python-decouple
- Pillow (future use)

### Deployment

- Docker
- Docker Compose

---

## 6. API Design

### Authentication

| Endpoint | Method | Description |
|---|---|---|
| `/api/auth/token/` | POST | Obtain JWT access & refresh tokens |

---

### Event APIs

| Endpoint | Method | Description |
|---|---|---|
| `/api/events/` | POST | Ingest a new security event |

**Behavior** - High/Critical severity events automatically generate alerts

---

### Alert APIs

| Endpoint | Method | Description |
|---|---|---|
| `/api/alerts/` | GET | List alerts (filter, paginate) |
| `/api/alerts/<id>/` | PATCH | Update alert status |

---

### Documentation

| Endpoint | Description |
|---|---|
| `/swagger/` | Swagger UI |
| `/redoc/` | ReDoc UI |

---

## 7. Business Logic

- **Automatic Alert Creation** using Django signals
- **Role-Based Permissions**
  - Admin: Full access
  - Analyst: Read and update alerts
- **Status Management**
  - `Resolved` status automatically sets `resolved_at`
- **Optimized Queries** to prevent N+1 issues

---

## 8. Security Model

- JWT Authentication with refresh tokens
- Custom permission classes
- Input validation and serializer enforcement
- ORM-based SQL injection protection
- CORS headers and Django security middleware
- Secure defaults for all endpoints

---

## 9. Environment Configuration

Example `.env` file:

```
DEBUG=True
SECRET_KEY=your-secret-key
DATABASE_URL=sqlite:///db.sqlite3
ALLOWED_HOSTS=127.0.0.1,localhost
```

---

## 10. Docker Support

The project includes Docker and Docker Compose configuration for easy deployment.

```
docker-compose up --build
```

---

## 11. Testing

- Unit tests for models, APIs, and permissions
- Django test framework
- Coverage-focused testing for critical logic

---

## 12. Future Enhancements

- WebSocket-based real-time alerts
- Email / SMS / Webhook notifications
- SIEM integrations
- Alert correlation and deduplication
- Dashboard frontend (React / Vue)

---

## 13. License

This project is released for educational and portfolio purposes.

---

**Author:** Vicky Kumar